AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 7

Design Problem: An LLM Gateway

Forty teams share one provider account. Design the component that gives them keys, budgets, routing and failover, passes streams through untouched, and doesn't become the reason all forty go down at once.

Module	AI10 of 13
Lesson	7 of 7, design problem
Prerequisites	All of AI10, AI08 L2, AI09 L4, HLD M04, HLD M11
Exam relevance	Out of scope as a design
Reading time	~50 minutes

Contents

1. Design an LLM Gateway
2. Module Revision

Lesson 7 of 7 · AI Production Engineering

Design an LLM Gateway

Key idea

Reserve the worst case before calling, settle the real cost afterwards, relay every event immediately, and fail over only before the first byte. Treat what passes through as opaque. Keep the gateway stateless, so that its only hard dependency is a limit store, and decide in advance what happens when that fails.

> *Before: all of AI10. Teaches: applying the module, using the ten-part template. Used by: AI12's gateway settings, and system design interviews.*

This is a recommended reference design, not a specific company's system. All traffic figures are assumptions. I built a small working version, `GatewayLab.java`, against a local fake server, and its limits are stated with its results.

1. Scope

In scope	Out of scope, on purpose
Credentials and attribution	Prompt management
Rate limits and budgets, per team and tenant	Agent orchestration
Routing and failover	Retrieval
Streaming pass-through	Guardrail content policy, which plugs in (AI09 L5)
Usage metering and audit	Training or hosting models

First question: do you need one at all? L2's alternatives apply. For this problem, we assume forty teams and a security requirement to remove provider keys from services, which justifies it.

2. Requirements

Type	Requirement	Target
Functional	Anthropic-format API, so existing SDKs work	Change the base URL only
Functional	Per-team budgets, and per-tenant fairness	Hard limits, with a small bounded overshoot
Functional	Attribute usage	By team, tenant, feature and key
Functional	Failover	Before the first byte
Non-functional	Added latency	Tens of milliseconds at p95, to be measured
Non-functional	Availability	Higher than the provider's, as it shouldn't be the weak link
Non-functional	New API features	Work with no gateway change
Security	Holds all prompts	No content logged by default

3. Estimation

All inputs are assumptions. Prices are Sonnet 5.5's, for simplicity.

Quantity	Calculation	Result
Requests per day	Assumed	5,000,000
Average rate	5M ÷ 86,400	57.9 per second
Peak, at 5×		**about 289 per second**
Concurrent streams at peak, 12 seconds each	289 × 12	**about 3,468**
Limit store operations, 2 per request	289 × 2	**about 578 per second**
Usage records, 500 bytes each	5M × 500 B	2.5 GB per day
Full content, if retained, 20 KB each	5M × 20 KB	100 GB per day

Per request: 2,000 uncached input, 2,000 cached, 400 output.

Item	Calculation	Cost
Uncached input	2,000 × $2 / M	$0.0040
Cache read	2,000 × $0.10 / M	$0.0002
Output	400 × $10 / M	$0.0040
Per request		**$0.0082**
Per day	× 5M	**$41,000**
Per month, 30 days		**about $1,230,000**

Now compare with provider limits. This step gets missed, and it changes the design.

Limit, at peak	Our need	Scale tier, Sonnet 5.5
Requests per minute	289 × 60 = **17,340**	13,000
Uncached input per minute	17,340 × 2,000 = **34.7M**	10M
Output per minute	17,340 × 400 = **6.9M**	2M
Monthly spend	**$1.23M**	$200,000 cap

This traffic is beyond the top standard tier on every line, so it needs a Custom tier, arranged in advance. Routing a share to Haiku helps, as limits are per model class, and caching more helps input. That's a sales conversation and an architecture decision, and neither is solved by a bigger gateway.

4. API

The gateway exposes the provider's own format, so clients need only a new base URL and key.

```
POST /v1/messages
x-api-key: gw_team_payments_...
anthropic-version: 2023-06-01
anthropic-beta: <forwarded verbatim>
x-gateway-tenant: acme
x-gateway-feature: invoice-summary
```

Header or field	Treatment
`x-api-key`	**Replaced** with the real credential. Never forwarded
`anthropic-version`, `anthropic-beta`	Forwarded unchanged
Body	Opaque, except for reading `model` and `max_tokens`
`model`	May be a logical name, such as `support-classifier`, mapped by config
`x-gateway-*`	Attribution, consumed and not forwarded

Tenant identity: when a team's service makes the call, `x-gateway-tenant` is that service's claim. The gateway trusts it only for attribution and fairness among that team's tenants. Don't treat it as an access control (AI09 L4).

Errors:

Case	Response
Unknown key	401, in the same error shape
Budget used up	429, with `retry-after` in integer seconds
Upstream error	Body forwarded unmodified, plus `retry-after` and `x-should-retry`
Every route failed	Last upstream error, unmodified

5. Data model

```
CREATE TABLE gateway_key (
  key_hash     TEXT PRIMARY KEY,   -- never the key itself
  team_id      TEXT NOT NULL,
  revoked_at   TIMESTAMPTZ
);

CREATE TABLE route (
  logical_name TEXT NOT NULL,
  version      INT NOT NULL,
  targets      JSONB NOT NULL,     -- ordered list: platform, model, parameters
  eval_run_id  TEXT NOT NULL,      -- the evidence for this route
  PRIMARY KEY (logical_name, version)
);

CREATE TABLE usage_event (
  request_id   TEXT PRIMARY KEY,   -- deduplicates replays from the queue
  team_id      TEXT NOT NULL,
  tenant_id    TEXT,
  feature      TEXT,
  model        TEXT NOT NULL,
  input_tokens BIGINT, cache_write BIGINT, cache_read BIGINT, output_tokens BIGINT,
  cost_micros  BIGINT NOT NULL,
  estimated    BOOLEAN NOT NULL,   -- true when the stream ended before final usage
  stop_reason  TEXT,
  created_at   TIMESTAMPTZ NOT NULL
);
```

These schemas haven't been run. Budgets and rates live in the limit store, as key-value counters. `estimated` matters because a disconnected stream may never report final usage.

6. High-level design

Baseline. Each service holds the provider key.

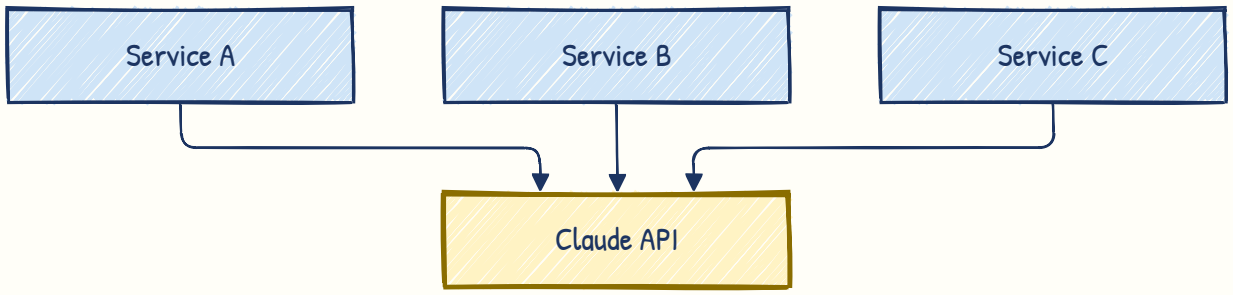


Figure 1. Baseline. Simple, and nothing can be attributed, limited or revoked in one place.

Improved. One gateway. New problem: it's now a single point of failure, and its budget checks race.

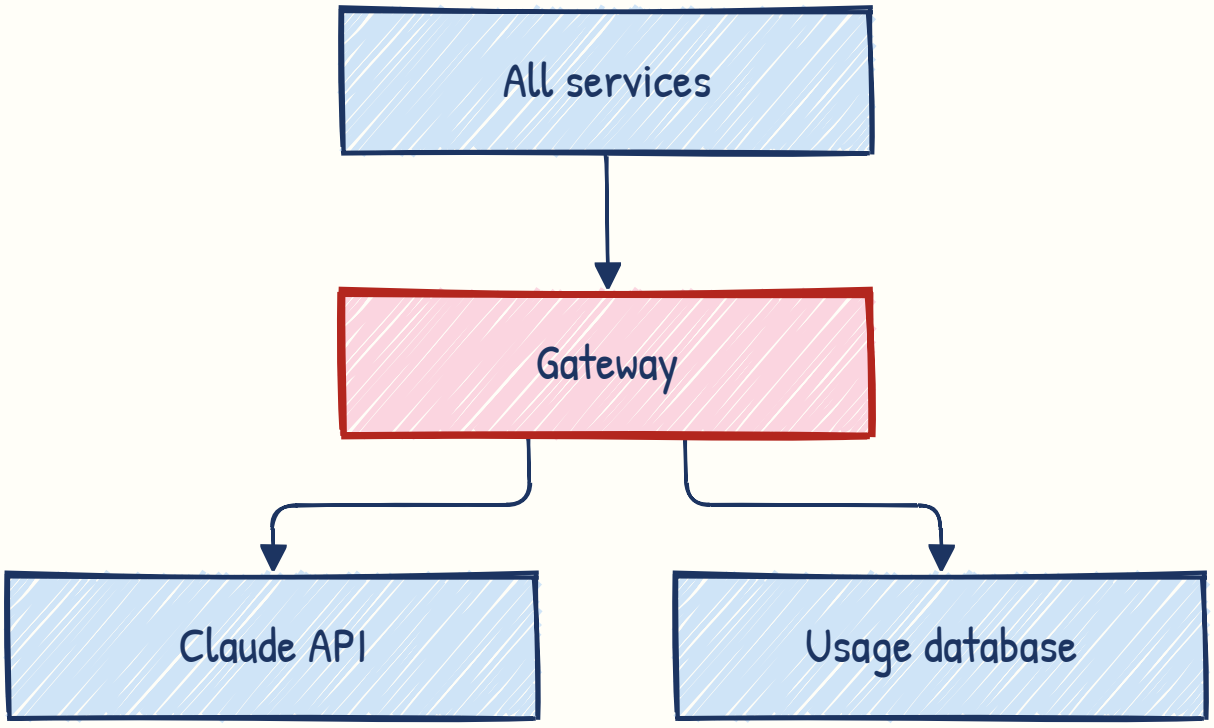


Figure 2. Improved. Central keys and metering, but one instance, and a budget check that many concurrent calls pass at once.

Production.

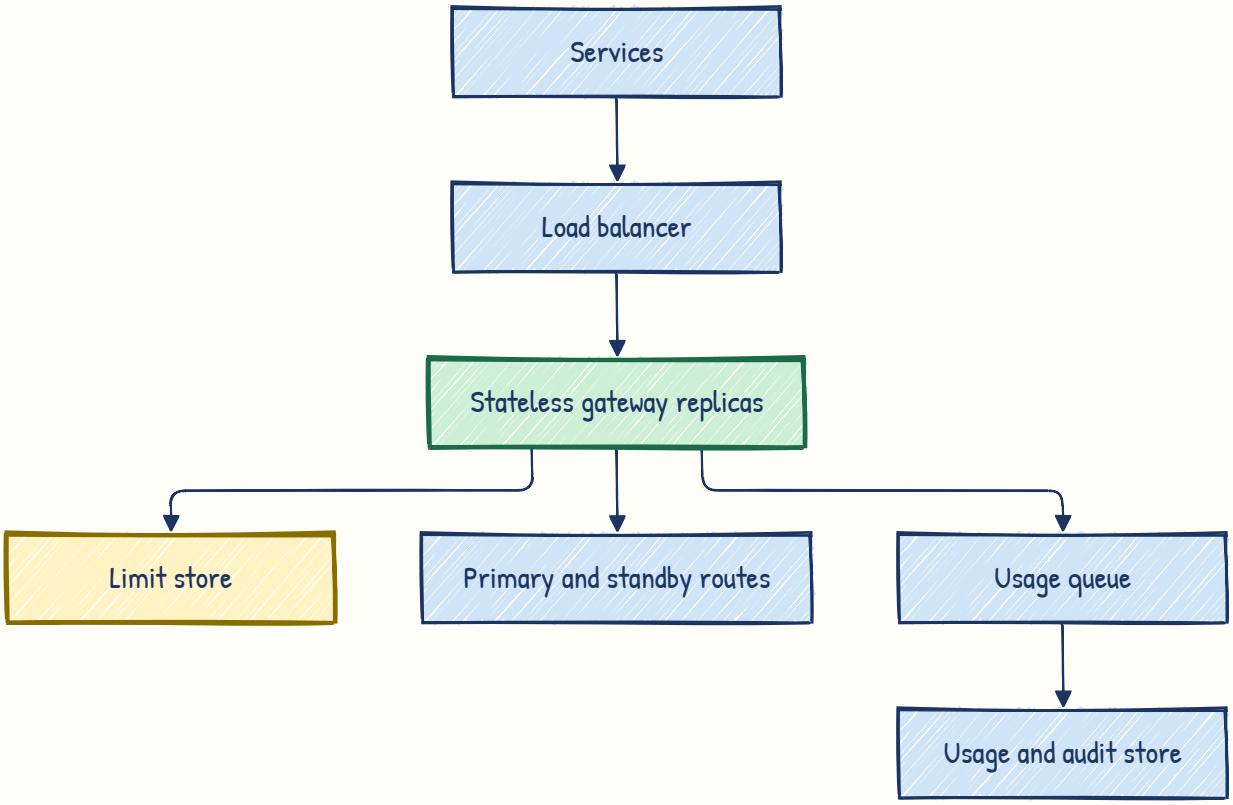


Figure 3. Production. Replicas are stateless. Usage leaves through a queue, so that a slow database never delays a request.

Component	Why it exists	How it communicates
Load balancer	Spreads streams. It must not buffer	HTTP, with long timeouts
Gateway replicas	Auth, limits, routing and relay	Stateless. Config cached locally
Limit store	Budgets and rates across replicas	Redis or similar, with atomic operations
Route config	Logical name to targets	Versioned, pushed or polled
Secret manager	Real credentials	Fetched and rotated
Usage queue	Keeps metering off the request path	Kafka or similar
Usage store	Attribution and invoices	Warehouse
Reconciler	Compares with provider's report	Daily job (L4)

Request flow

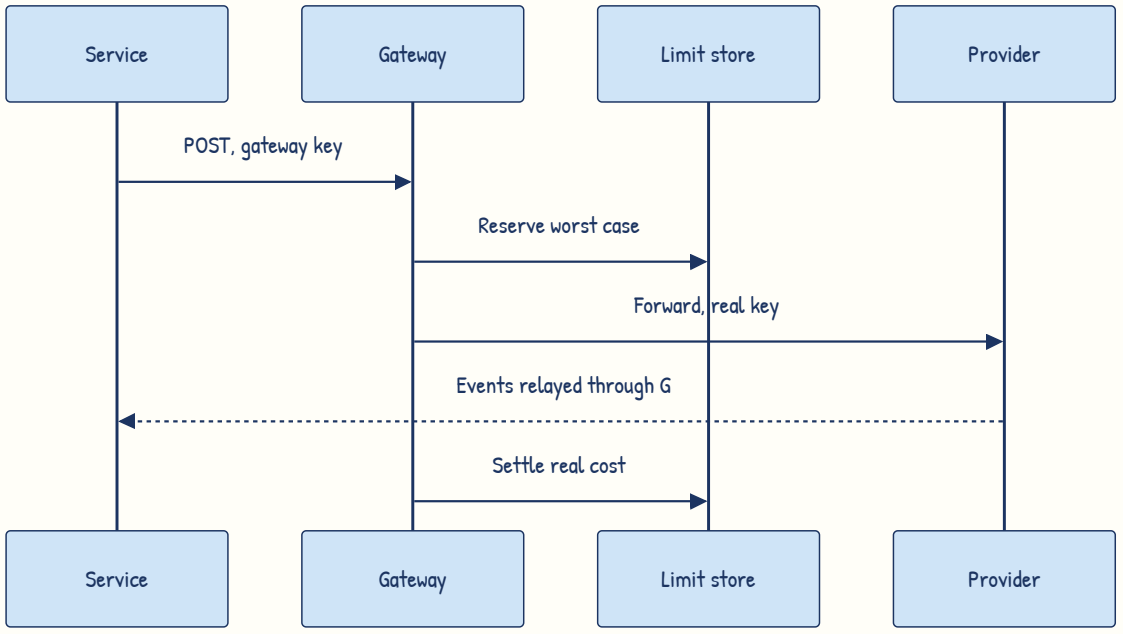


Figure 4. Reserve, forward, relay, settle. Metering happens after the stream ends, and never delays the first token.

Step	Detail
Authenticate	Hash the key, and look it up in a local cache
Reserve	`max_tokens` output plus estimated input, at this model's price
Route	Logical name to first healthy target
Forward	Replace the key, and keep everything else
Relay	Flush each event as it arrives
Meter	Read `usage` from `message_start` and `message_delta`
Settle	Release the reservation, and commit the real cost
Record	Send a usage event to the queue

7. Deep dives

Deep dive 1: budgets under concurrency

Cost is unknown until the stream ends, and many requests run at once. So a naive check can pass for all of them. Real output, from real threads, with 200 concurrent calls that each cost $0.05 against a $1 budget:

```
The check-then-spend race, 200 concurrent calls, $1 budget:
  check, call, then add: admitted 200, spent $10.00
  reserve, call, settle: admitted  20, spent $1.00
```

Ten times the budget, in an instant. **Reserve the worst case first, then settle down to the real cost.**

```
if (committed + reserved + amount > limit) {
    return false;
}
reserved += amount;
```

It's conservative, and here's its cost. Real output, from my gateway, with a budget of $0.015:

```
Authentication and budget:
  unknown key: 401, upstream calls 0
  acme, budget $0.015, six calls: [200, 200, 200, 200, 429, 429], committed micro-dollars 8120
  globex at the same moment: 200
```

Calls were refused with $0.00688 left in the budget, since each reservation is about $0.00705 for its `max_tokens`. A generous `max_tokens` holds more budget, so reservations for very large values can starve a team. Accept that, or cap the hold and accept a bounded overshoot. Note also that an unknown key never reached the upstream, and that one tenant's exhausted budget didn't touch another.

Across replicas. One central counter means a network call per request. At 578 operations a second, that's fine for Redis in our estimate. At greater scale, replicas lease budget in chunks, and spend locally:

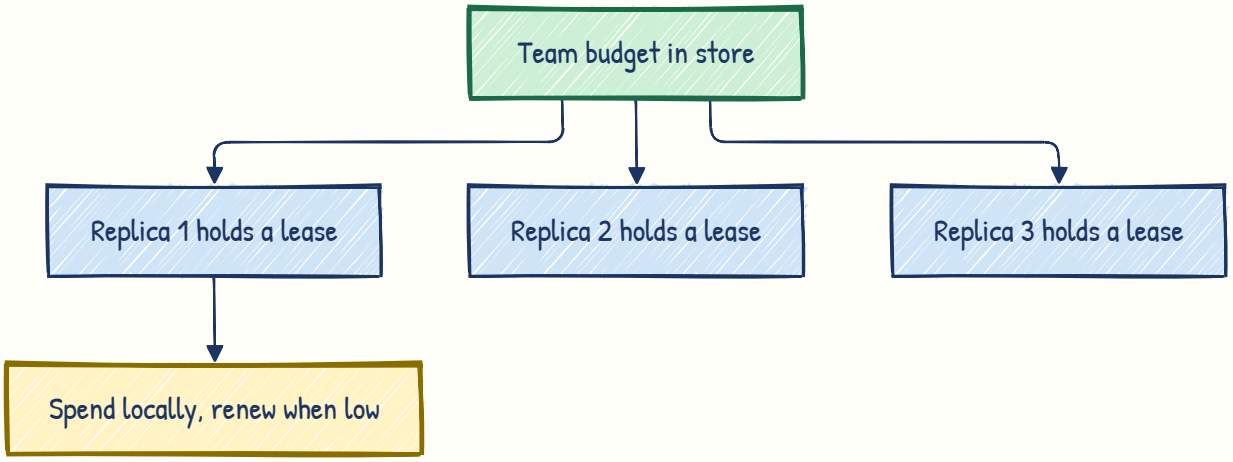


Figure 5. Leases cut calls to the store. The price is a bounded overshoot, up to one lease per replica.

Dead replicas strand their leases, so give them an expiry.

Deep dive 2: streaming

The gateway's rules come directly from Claude Code's compatibility guide (L2). Real timings, on a laptop, against a local fake:

```
20 events, 10 ms apart, 30 runs, median ms, one laptop:
  direct:         first text  20, whole stream 240
  through relay:  first text  26, whole stream 249
  buffering gateway: first text 247
```

These are in-process HTTP on loopback, so production overhead will differ. The relay itself is small. **Any hop can buffer:** the load balancer, a service mesh, or a compression filter. Measure first-token time from the client, and alert on it.

When the client disconnects, stop. Real output, where the client leaves after the first text event of a 200-event stream:

```
Client disconnects after the first text event:
  events the upstream generated, of 200: 4, upstream saw cancel: 1
  ledger: [acme input=1000 output=3 ESTIMATED, client disconnected cost=2030]
```

Closing the upstream connection stopped generation in my fake. **Whether a real provider stops, and what it bills, needs checking with real traffic, so I can't state that.** Mark the record `estimated`, as final usage never arrived, and reconcile it later.

Failure mid-stream. Real output for the part that can be handled:

```
Failover:
  primary 529: status 200, primary calls 1, secondary calls 1, failovers 1
  both 529: status 529, body forwarded: true, held now 0
```

The client saw a success in the first case, and never knew. In the second, it received the upstream's own error, and the reservation was released, with `held now 0`. After the first byte, there's no invisible failover, as L1 explained.

```
if (response.statusCode() < 500 || i == upstreams.size() - 1) {
    break;
}
```

Deep dive 3: when the gateway's own parts fail

The gateway mustn't be less reliable than what it fronts.

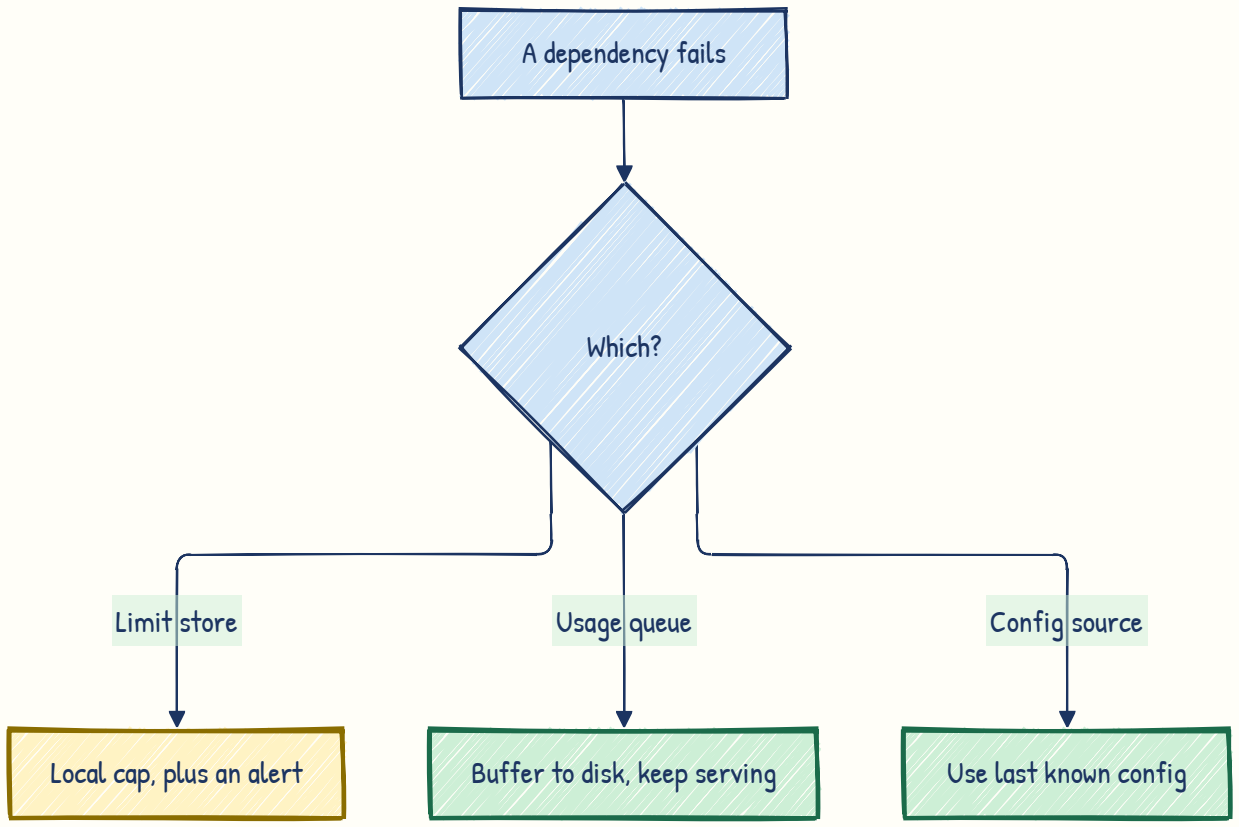


Figure 6. Each dependency has a decided behaviour. The request path never waits on the usage store.

The bypass question. Should critical services be able to skip the gateway in an emergency? That undoes key control, so it needs a break-glass credential, with an alert when it's used. Decide this in peacetime.

Deep dive 4: security

Risk	Control
All prompts pass through	No content logs by default. Restricted, short-lived sampling (AI09 L4)
The real keys	Secret manager, rotation, never in logs
Stolen gateway key	Hash at rest, revoke fast, per-key limits, anomaly alerts
Attacker uses it as an open proxy	Auth everywhere, and an allowlist of upstream hosts
A team claims another's tenant	Keys scoped to team. Tenant header for attribution only
Cached answers across tenants	Scoped keys, if a cache exists at all (L5)

8. Scaling

Pressure	First response	Later
Concurrent streams	Replicas. Virtual threads make long waits cheap (AI08 L2)	Shard by team
Limit store load	Pipelining	Budget leases
Provider limits	Custom tier, more caching	Model routing, more platforms
Usage volume	Queue and batch writes	Partitioned warehouse
Big-team noise	Per-team queues	Dedicated replicas or workspace

How many replicas? It depends on your runtime and your own load test. At an assumed 1,000 concurrent streams per replica, peak needs 4, and I'd run 8 or more across zones to survive a zone's loss. I haven't load-tested this gateway, so that per-replica figure is only an assumption. AI08's stream lab has some local evidence for virtual threads, on one laptop.

Deploys. Streams last minutes, so a rolling restart must drain connections. Stop sending new requests to a replica, and wait for current streams, up to a limit.

Failure handling

Failure	Detection	Response
Replica crash	Health checks	In-flight streams fail, and clients retry. Reservation expires
Limit store down	Timeouts	Local cap per replica, and an alert
Provider overload	529s	Breaker, then standby route
Spend cap at provider	`enforced_spend_limit_reached`	**Page.** Failover won't help on the same account
Standby rejects a parameter	400s after failover	Per-route parameters, tested in CI
Usage queue down	Producer errors	Local buffer, and never block requests
Reconciliation mismatch	Daily job	Investigate estimates and double counts
Stale reservation	Hold count grows	Expiry on reservations
New API feature fails	Client errors	Forward unknown fields, and run the newest SDK against staging

9. Trade-offs

Decision	Choice	Alternative	Why
Client interface	Provider's format	Own abstraction	SDKs and new features work. It costs provider independence
Body handling	Opaque	Fully parsed	Survives API changes
Budget check	Reserve and settle	Add after	Check-then-spend overspent 10×
Metering	Asynchronous	Inline	First-token time stays clean
Failover	Before first byte	Mid-stream	User has already seen text
Limit store failure	Local cap	Fail closed	A counter shouldn't stop all AI
Content logs	Off by default	On	It's a big privacy risk
Build or buy	Depends on needs		L2. I evaluated no products

When this is over-built: at one or two teams, a small library and workspaces do most of it. At this size, a product may also be a better answer than building.

10. Follow-up questions

Q How would you support Bedrock as a standby, when its request format differs?

A Claude Code's guide lists separate endpoints and formats for each platform. The gateway would translate between formats on that route, which breaks the "opaque" rule for it, and event streams differ in format as well. Keep this translation in a dedicated adapter, and test feature coverage per route, as some beta features won't exist there.

Q Why not deduct the budget at the end of each request only?

A Hundreds of concurrent requests would all pass the same check before any is counted. My race test overspent by a factor of ten. Reservation closes that gap, at the price of holding budget for the duration.

Q A team sets `max_tokens` to 128,000 on every request. What happens?

A Their reservations become very large, at $1.28 of output each on Sonnet 5.5, so they'd exhaust their own budget's headroom, and see 429s with money left. OTPM isn't affected by the setting, as the rate limits page says, but your gateway's hold is. Options: cap the hold, with bounded overshoot, or tell the team. The fix costs them nothing.

Q How do you stop the gateway becoming the place where all policy accumulates?

A Keep product logic in the services. The gateway owns what's common to all: identity, limits, routing and metering. Guardrails can plug in by route, with an owner per route. A code review rule helps: anything needing to parse a prompt's meaning needs an explicit decision.

Q How do you bill tenants accurately when some records are estimates?

A Keep the `estimated` flag. Reconcile totals with the provider's usage report per key or workspace, and apply the difference proportionally, or absorb it. Publish that policy. Give each big tenant its own workspace if they need exact attribution.

Certification lens

How this lesson maps to CCAR-F

Out of scope as a design. Gateways aren't in the guide. The practical overlap is Claude Code's gateway settings (AI12) and handling of API errors. Use this lesson for architecture interviews and real work.

These are original practice questions, not real or leaked.

Q Q1 · Your gateway adds 20 ms at p50, yet users report answers appearing all at once after a long wait. What would you check?

A Buffering at some hop: a load balancer, a service mesh, a compression filter, or a framework's response buffer. Each makes first-token time equal to total time. My buffered relay showed exactly that shape. Measure first-token time at the client and at each hop.

Q Q2 · Failover to a second platform works in a drill, and during a real outage it fails with 429s. Give two reasons.

A The standby's limit may be sized for a drill and not for full load. A sudden increase can also trigger acceleration limits, as the docs warn. A cold prompt cache also raises uncached input tokens, which count toward ITPM. Keep some live traffic there, and size the limit for full load.

Q Q3 · Why does the estimation step compare peak traffic with provider limits?

A Because it can show that a design can't run at all on a standard tier. Ours needed about 17,340 requests a minute, 34.7M input and 6.9M output per minute, and $1.23M a month, all beyond the Scale tier on Sonnet 5.5. No gateway fixes that. It needs a Custom tier, more caching, or routing a share to another model class.

Revision - Lesson 7

→ Reserve the worst case, relay events immediately, settle afterwards, record asynchronously.

→ Check-then-spend overspent by ten times in a real race test.

→ Reservations hold budget. A large `max_tokens` holds more. Leases scale it, with bounded overshoot.

→ Keep the body opaque. Replace the key. Forward beta headers and errors unchanged.

→ Fail over before the first byte only. After it, tell the client.

→ Stop the upstream when a client leaves, and mark that record as estimated.

→ Compare peak needs with provider limits. Ours exceeded the Scale tier on every line.

→ Decide each dependency's failure behaviour in advance. Usage must never block requests.

→ All prompts pass through, so no content logs by default.

Your notes

Module Revision

Self-assessment

#	Can you...
1	Tell a rate limit, a spend cap and your own spend limit apart?
2	Give the response to each error status?
3	Explain why cache reads raise throughput?
4	Explain what backoff and a breaker each do in an outage?
5	Explain why retries should live in one layer?
6	Name four timeouts, and what each catches?
7	Say what to do when a stream fails after the first byte?
8	Explain why a fallback model needs evals?
9	List what a gateway provides, and its cost?
10	Give the pass-through rules, and why an allowlist fails?
11	Compare routing strategies?
12	Calculate cost per completed task, including rework?
13	Define task, trial, grader, transcript, outcome and suite?
14	Distinguish capability from regression evals?
15	Explain why you grade outcomes rather than paths?
16	Show how an average hides a weak segment?
17	Explain calibration, and how to choose a threshold?
18	Explain why a global threshold can leak a weak segment?
19	Explain stratified sampling, and its re-weighting?
20	Estimate what a suite's size can detect?
21	Explain why a fixed gate on few tasks misleads?

#	Can you...
22	Validate a model judge?
23	Measure hallucination, including abstention?
24	Find problems in an agent's trace?
25	Calculate total input tokens and hit ratio?
26	Avoid double counting from OpenTelemetry spans?
27	Compare head with tail sampling?
28	Put cost levers in order, and say why model choice is last?
29	Explain why a one-shot call shouldn't use caching?
30	Judge a semantic cache by the cost of wrong answers?
31	List what counts as a release in an AI system?
32	Describe the stages, and shadow's limit with write tools?
33	Size a canary?
34	Give the model lifecycle states and notice period?
35	Show why one queue harms quiet tenants?
36	Correct an availability estimate for correlated failure?
37	List what surprises you in a failover?
38	Design budget reservation, and explain its cost?
39	Compare peak needs with provider limits?

Where this module is used

Idea	Where it returns
Inference serving and quantization	AI11
Gateways, OpenTelemetry and managed settings in Claude Code	AI12
TS 5.5, TS 4.5 and TS 5.3 questions	AI13

Working space